Lab 7: Build a Web Service Application with Elastic Load Balancing and Auto Scaling in a Cloud

A design of an elastic cloud-based application enables your application with high availability, better fault tolerance, auto scaling and load balancing in a cloud environment.

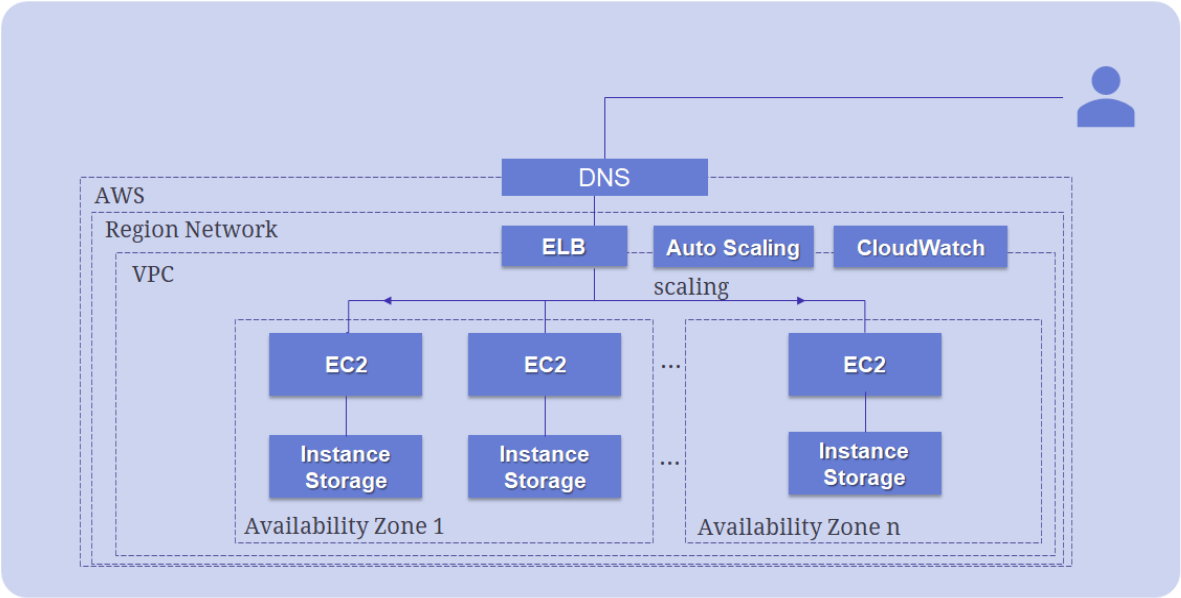
This lab session will let students explore using Amazon Elastic Load Balancing (ELB) and Auto Scaling services to support a web service application.

Topics covered

This lab will include two parts:

- Part 1, to launch an Amazon **Elastic Load Balancing (ELB)** service and build a simple web server application on top of multiple EC2 instances.
- Part 2 (optional), to launch and configure an Amazon **Auto Scaling** Service for the web application to auto scale in/out according to the demand.

Students need to complete Part 1 to build and run the web server in Amazon cloud and Part 2 is optional for autoscaling. A high-level system architecture is shown below.



A cloud architecture to auto scale a web application

Learning Objectives

- Creating an Amazon EC2 Web Server Image.
- Launching an Elastic Load Balancer (ELB)
- Verifying the load balancing
- Creating and Launching an Auto Scaling service
- Verifying the auto scaling
- Terminating EC2 instances, Elastic Load Balancer and Auto Scaling services

Duration

This lab takes approximately **100 minutes** to complete.

Part 1: Build a web server application using an Amazon Elastic Load Balancing (ELB) service

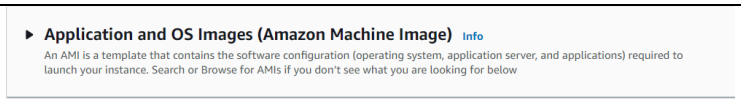
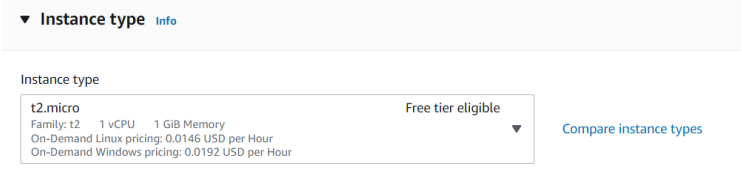
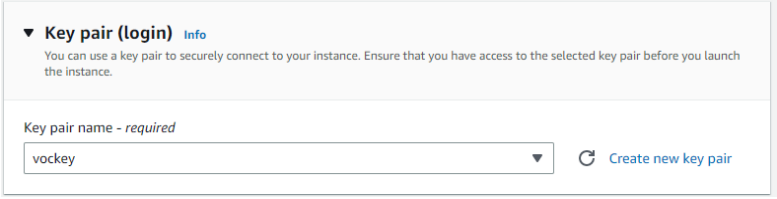
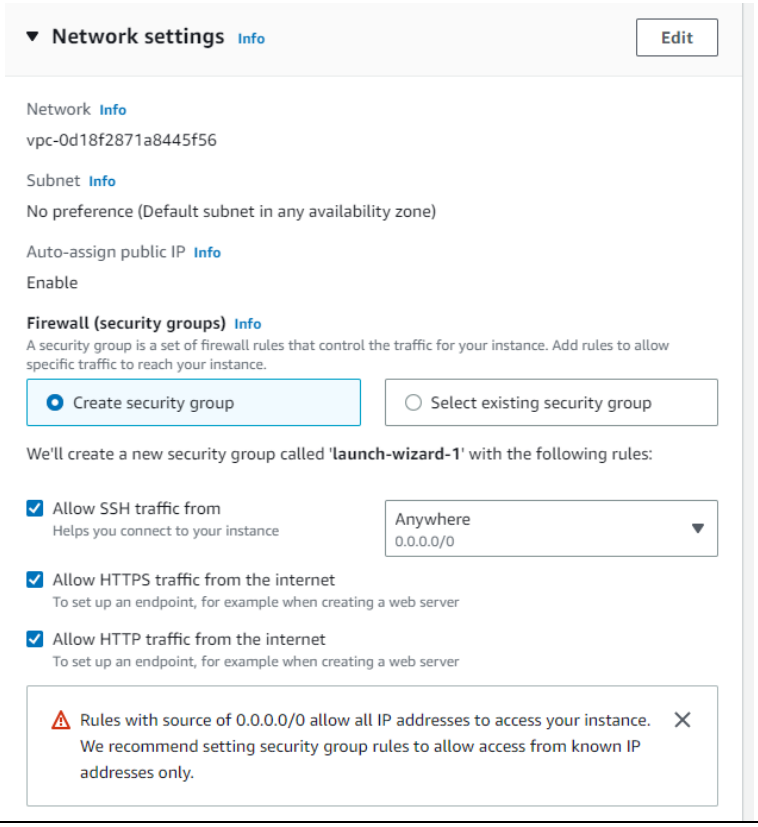
A cloud architecture with redundancy allows multiple servers to work collaboratively and dynamically balance the workload among multiple servers.

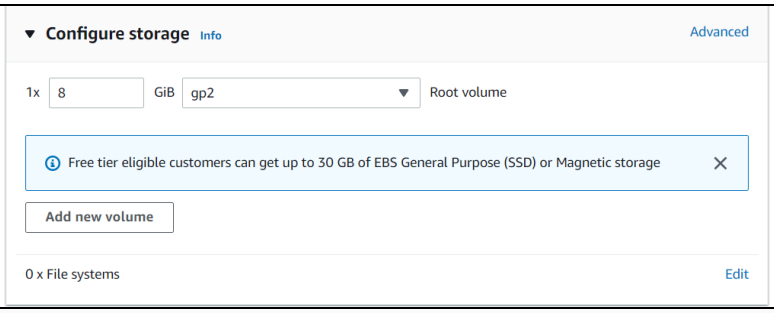
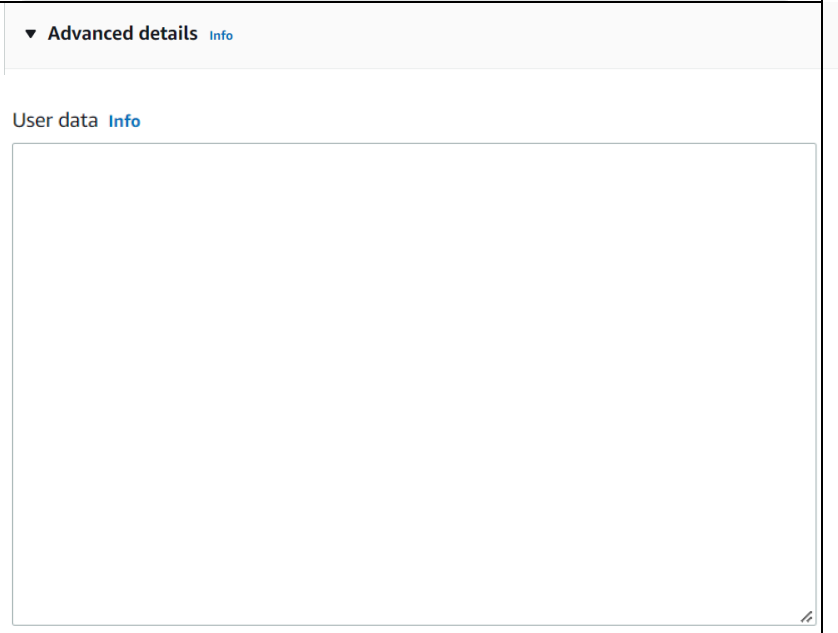
In Part 1, students can explore using Amazon **Elastic Load Balancing** service as an example to know how to automatically distribute incoming application traffic across multiple Amazon EC2 instances. Elastic Load Balancing helps to balance the workload and achieve fault tolerance by routing application traffic among multiple EC2 instances seamlessly.

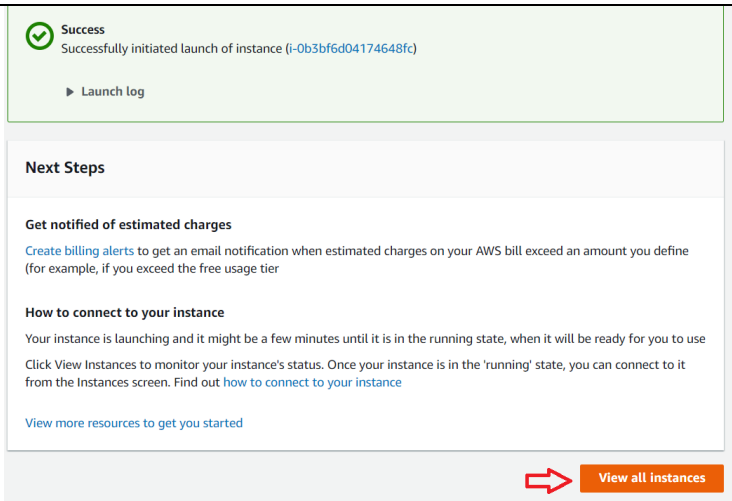
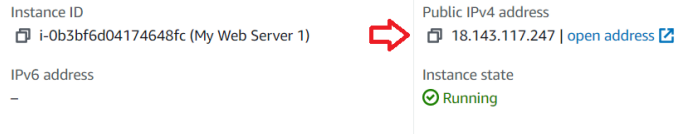
Task 1: Create Web Server Image and Launch Web Servers

1. Choose **Compute** in the search box to the right of **Services,** select **EC2,** and choose **Launch instances**

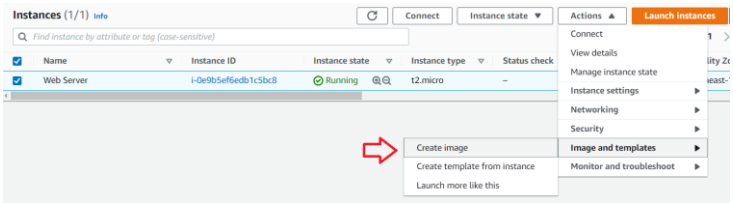
SN	Steps	Instructions
1)	Name the EC2 instance as "**WebServer1**"	

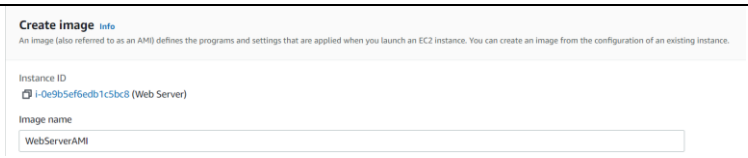
2)	Choose the default **AMI**	
3)	Choose the default instance type **t2.micro**	
4)	Choose the default key pair: **vockey**	
5)	**Network settings**	Create security group and select SSH, HTTPS, and HTTP traffic. 

6)	Configure storage	
7)	Expand advanced details to configure the user data	 Copy the following scripts into User data. #!/bin/bash # Update package and start Apache web server sudo yum update -y sudo yum install httpd -y sudo systemctl start httpd sudo systemctl enable httpd # Create a simple index.html file echo '<html><h1>Hello From Web Server 1!</h1></html>' > a.html sudo cp a.html /var/www/html/index.html rm a.html # Set appropriate permissions sudo chown apache:apache /var/www/html/index.html sudo chmod 644 /var/www/html/index.html # Display server status sudo systemctl status httpd The *user data* will perform automated installation and configuration tasks after the instance starts. The script will:

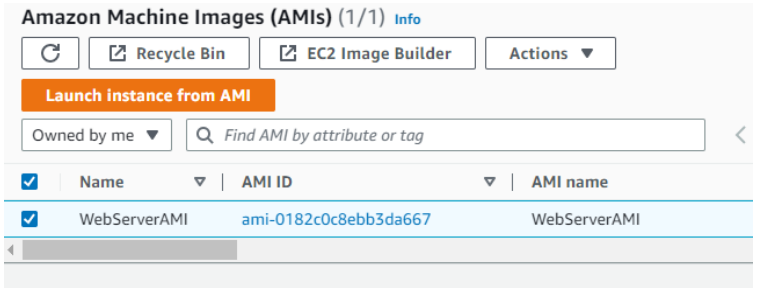
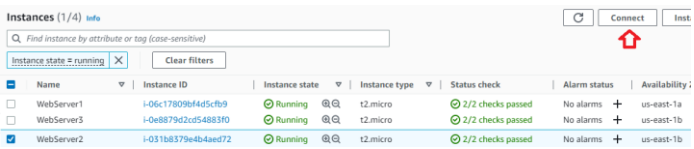
		○ Install an Apache web server (httpd) ○ Configure the web server to automatically start on boot ○ Run the Web server once it has finished installing ○ Create a simple web page
8)	Launch instance	
9)	View all instances	
10)	Check the status	 Wait until the instance has been successfully initialised and checked.
11)	Get the public Ip address	 Open a web browser and check the url http://<public_ip_address>

2. Create a web server image

SN	Steps	Instructions
1)	Create the web server Image	Select Web Server instance, From Action select **Image and template** and choose **Create image** 

2)	Name the image: **WebServerAMI**	 Click **Create image** to create the image.

3. Launch two more web servers based on the VM image created

SN	Steps	Instructions
1)	Launch EC2 instance based on the WebServerAMI image.	Select **AMIs** under **Images** in the left pane. Choose **WebServerAMI,** and click **Launch instance from AMI.** 
2)	Launch **WebServer2** and **WebServer3**	Name the server, select WebServerAMI as the image, choose default key, and configure the network settings with the same VPC and/or different subnet, Auto-assign public IP Enable, and **choose the existing security group that was just created for WebServer1**. Click **Launch instance** to launch WebServer2. Repeat the steps to launch WebServer3.
3)	Modify the web index files for WebServer2 and WebServer3.	Go to instances in the left pane, select WebServer1. Wait until the instance has been successfully initialised and checked, select Connect. 

		$ sudo vim /var/www/html/index.html
		Modify the index.html as
		`<html><h1>Hello From Web Server 1!</h1></html>`
		
		Repeat Step 3) to modify the index.html at WebServer2 as.
		`<html><h1>Hello From Web Server 2!</h1></html>`
		
		Repeat Step 3) to modify the index.html at WebServer3 as.
		`<html><h1>Hello From Web Server 3!</h1></html>`

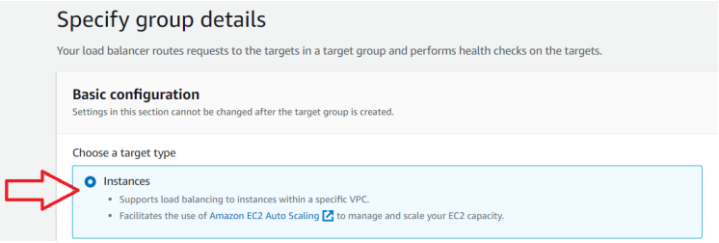
Open a web browser and check the url http://<public_ip_address> of WebServer2 or WebServer3. You should be able to view the webpages of the two servers.

Add the screenshots of WebServer2 and WebServer3 to Result 1 in the lab report

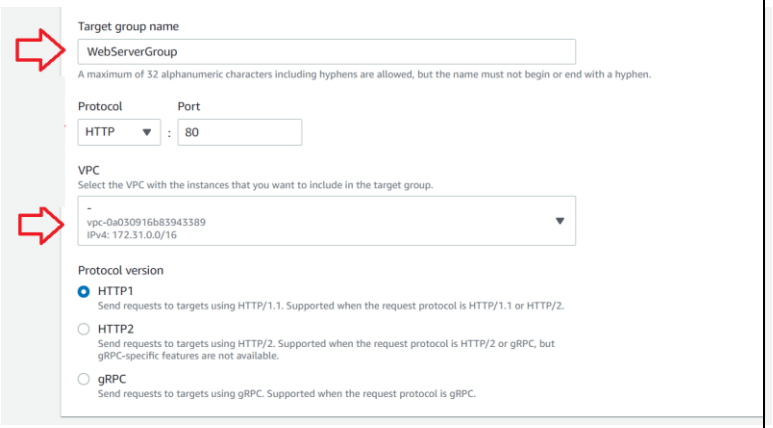
Task 2: Add and configure an Elastic Load Balancer (ELB)

You can follow the steps:

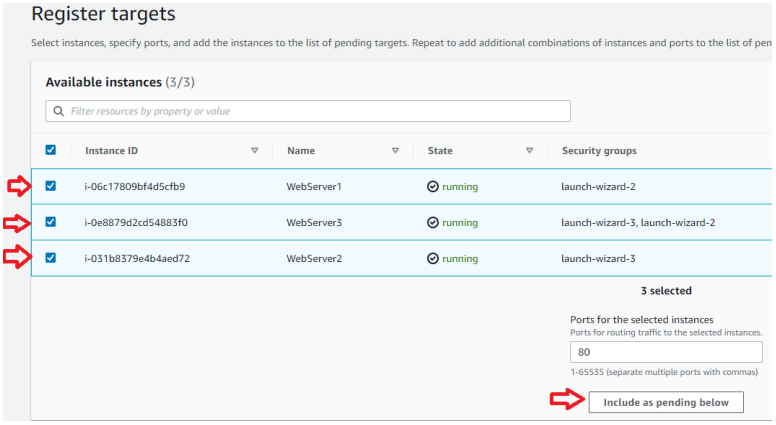
SN	Steps	Instructions
1)	Create a target group for load balancing	In the left navigation pane under **Load Balancing**, select **Target Groups**, and click **Create target group** ○ Choose a target type: **Instances**

<table>
<tr>
<td></td>
<td></td>
<td>


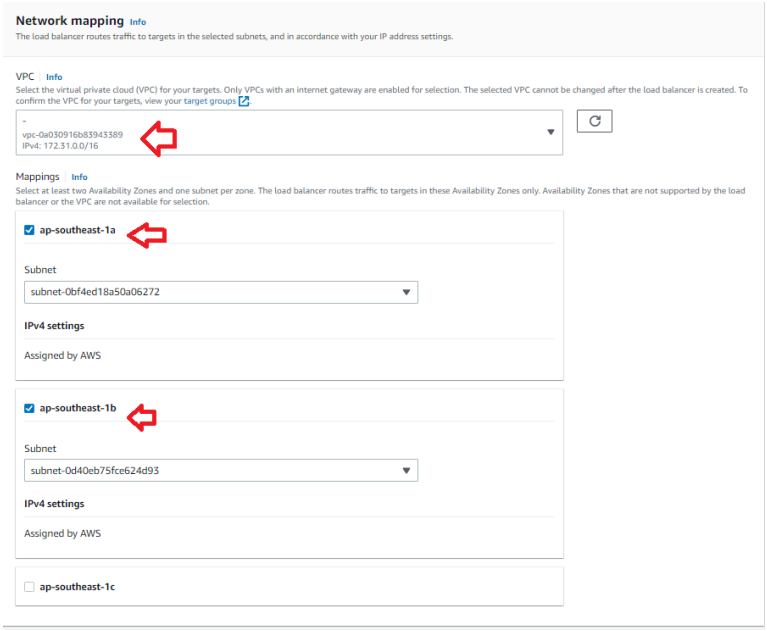
- Name target group: **WebServerGroup**
- Select **VPC** of web server instances from the **VPC** drop-down menu.

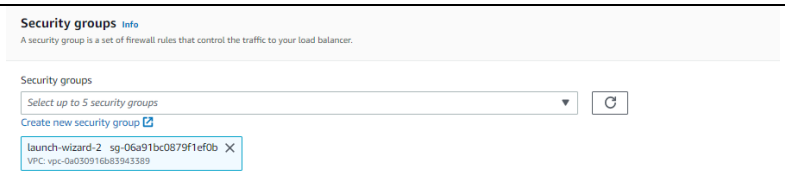


click **Next**
</td>
</tr>
<tr>
<td>2)</td>
<td>Register Targets</td>
<td>
Select WebServer1, WebServer2, and WebServer3 as the target.

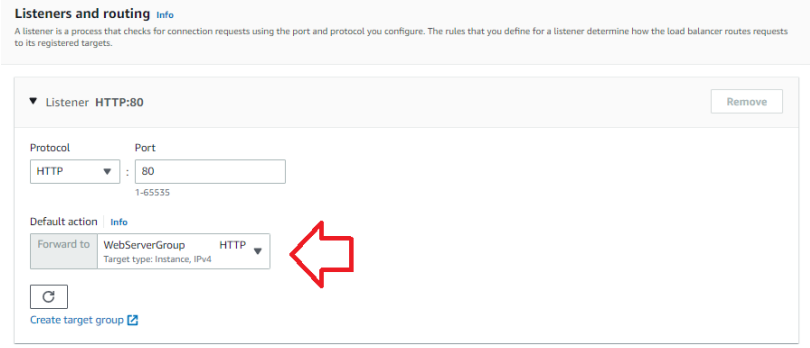


Click include as pending below and then include pending targets, the instances will be registered successfully.
</td>
</tr>
</table>

		Review the settings and choose **Create target group** **Add the screenshot to Result 2 in the lab report**
3)	Create a load balancer named **WebServerELB**	In the left navigation pane under **Load Balancing**, click **Load Balancers,** and click **Create Load Balancer**. ○ Under **Application Load Balancer**, choose **Create** ○ Under **Load balancer name**, enter: **WebServerELB** ○ Under Network mapping, choose VPC of the web server instances, and select at least **two subnets** and **the selected subnet should contain the registered web server instances** i.e., WebServer1, WebServer2, and WebServer3.  ○ In Security Groups, choose the security group as the same as the security group of web server instances and **remove the default security group** by clicking **X**.

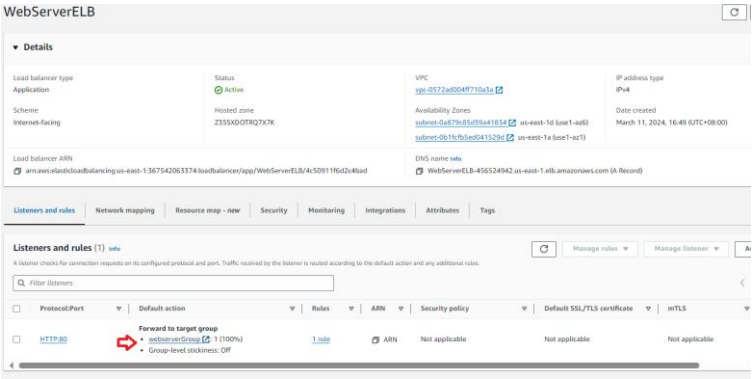


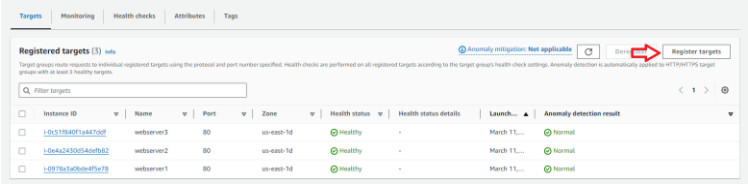
- For the Listener HTTP:80 row, set the Default action to forward to WebServerGroup.



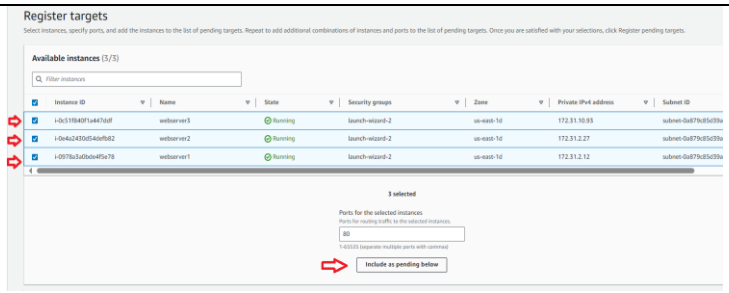
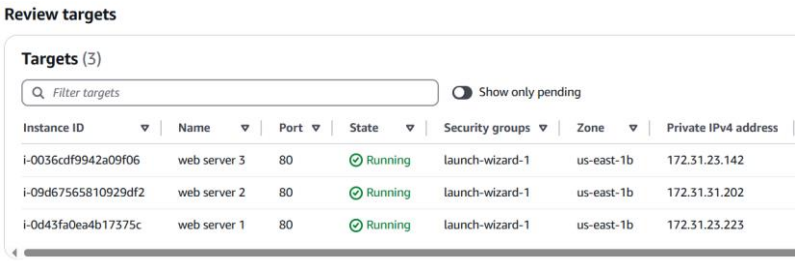
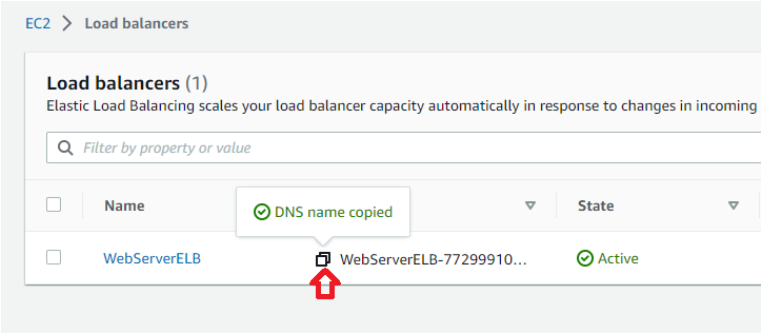
- Scroll to the bottom and choose **Create load balancer** and then click **View load balancer**

Check Registered Targets of WebServerELB and make sure the Targets have been registered.



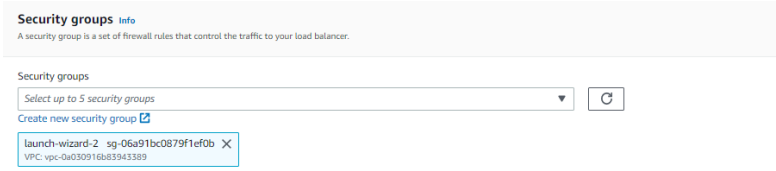


Register Targets

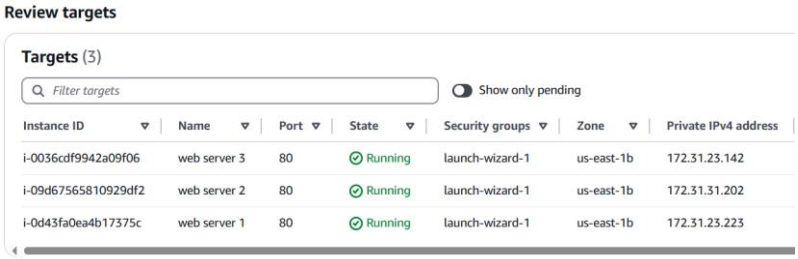
		 Wait for the load balancer to be ready until the status of the registered targets in the **WebServerGroup** target group shows Running.  **[Note]** Please double check if the subnet zones of the web servers 1 to 3 are inside the subnets of the load balancer WebServerELB. If not, edit the subnets of WebServerELB to include the subnet zones of the three web servers.
4	Test the load balancer	Get the DNS name of the WebServerELB, open a web browser, and access the url http://<WebServerELB_DNS_name>  Every few seconds, refresh the web browser and see if any changes. The load balancer is balancing the workload among the 3 servers.

Add the screenshot to Result 2 in the lab report.

Trouble shooting 1: If the load balancer DNS does not work, please check the security group of the load balancer. See if its security allows SSH, HTTP and HTTPS traffic. **Remove the default security group** by clicking **X** as it does not allow SSH, HTTP and HTTPS traffic.



Trouble shooting 2: If the load balancer DNS does not work even with the security group allowing SSH, HTTP and HTTPS traffic, please check if the instances have been successfully **registered** with Running status.



Sometimes, the instances are not registered properly because the subnets of the load balancer do not include the subnets where the instances were launched. If so, modify the subnets of the load balancer to include the subnets where the instances were launched.

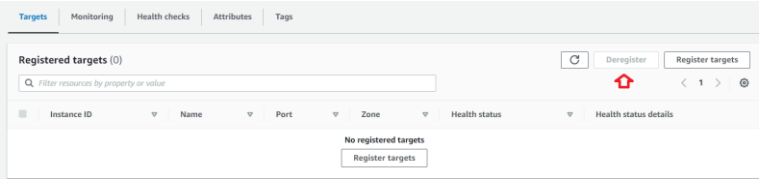
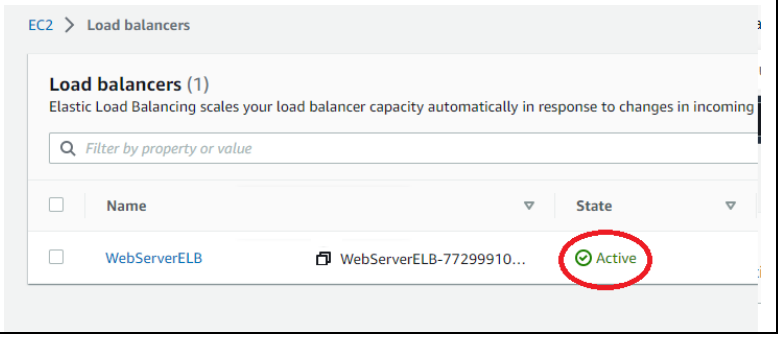
Part 2: Enable Auto Scaling for a Web Application

A cloud architecture with elasticity features enables **Auto Scaling** which improves the application availability and capacity according to user demands.

In Part 2, students can use Amazon Auto Scaling service as an example to configure your web application service to automatically increase capacity or the number of Amazon EC2 instances during demand spikes to maintain performance and decrease the capacity when there are low demands to reduce costs.

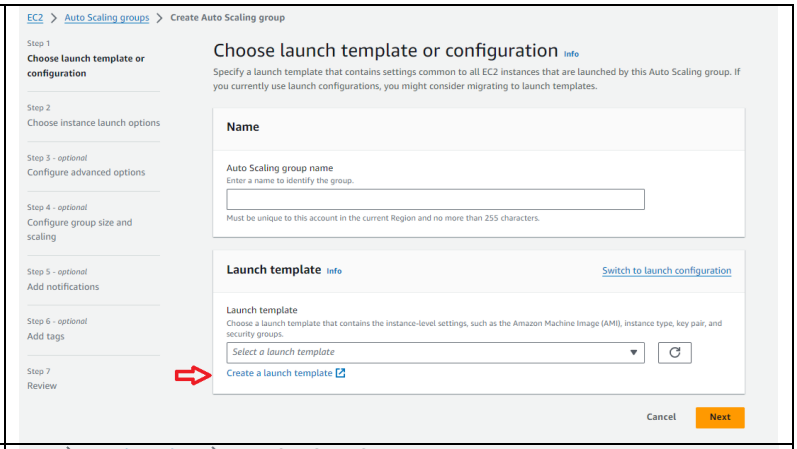
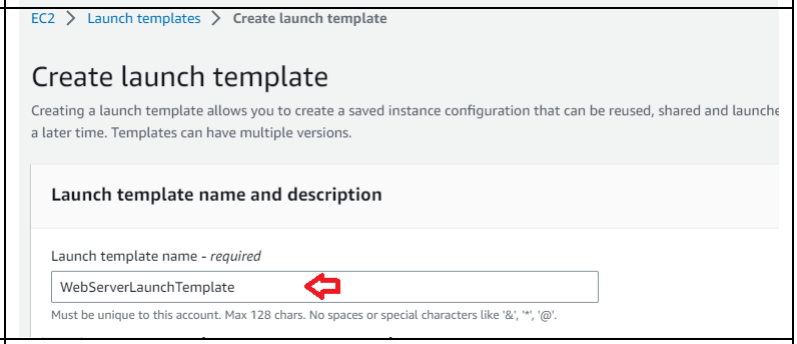
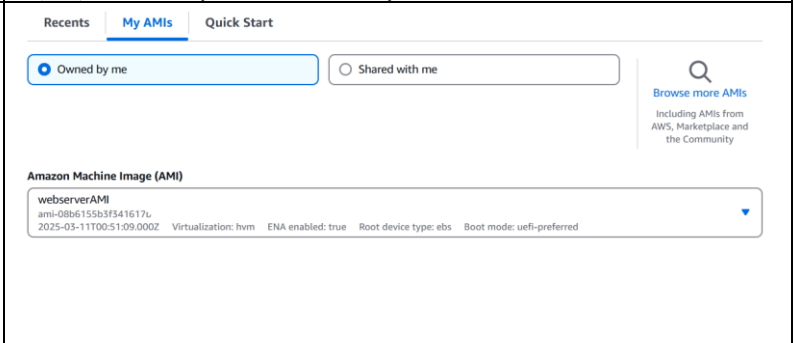
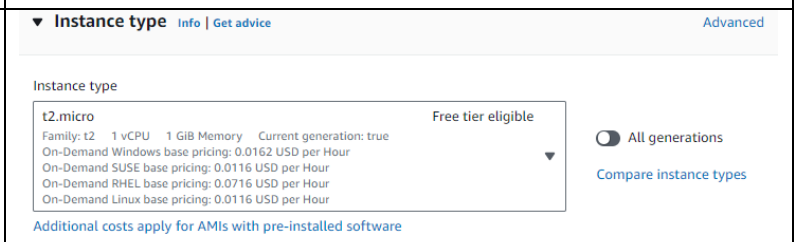
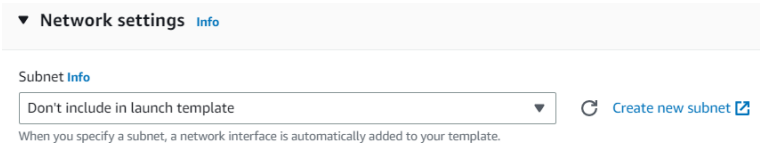
Task 3: Create and configure Auto Scaling Group

1. Prepare the Load balancer.

SN	Steps	Instructions
1)	Deregister the instances from the target group	Select Target Groups in the section of Load Balancing in the left pane, deregister the instances in the **WebServerGroup** target group if any. 
2)	Check the Load balancer if it is ready.	Check the load balancer if the status shows Active. 

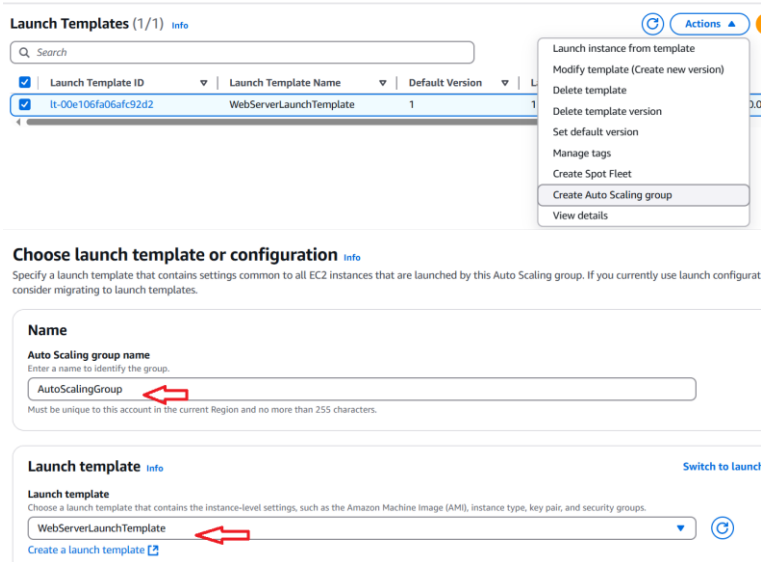
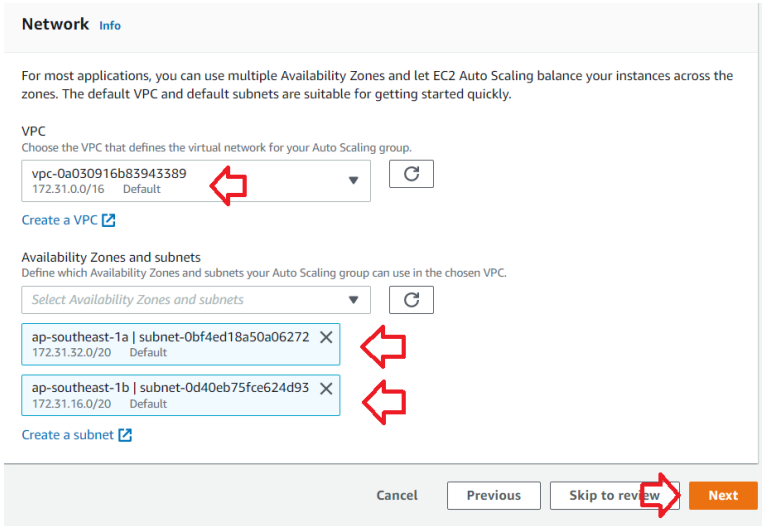
2. Create launch configurations for auto scaling. From the left navigation pane under **Auto Scaling**, choose **Auto Scaling Group**, and select **Create Auto Scaling Group**

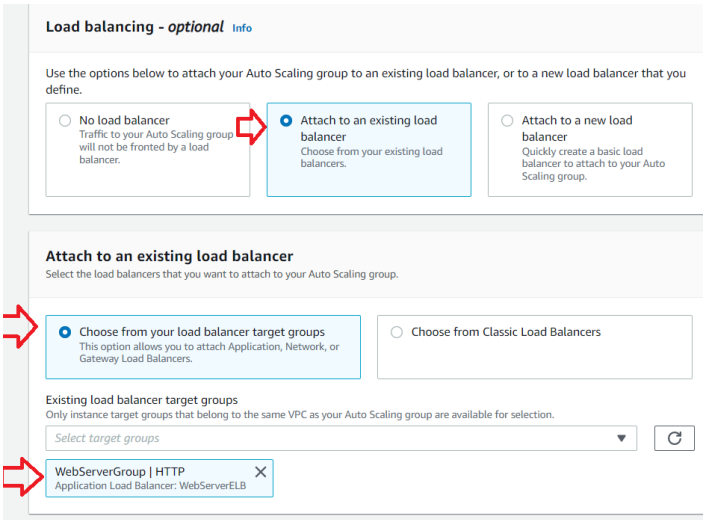
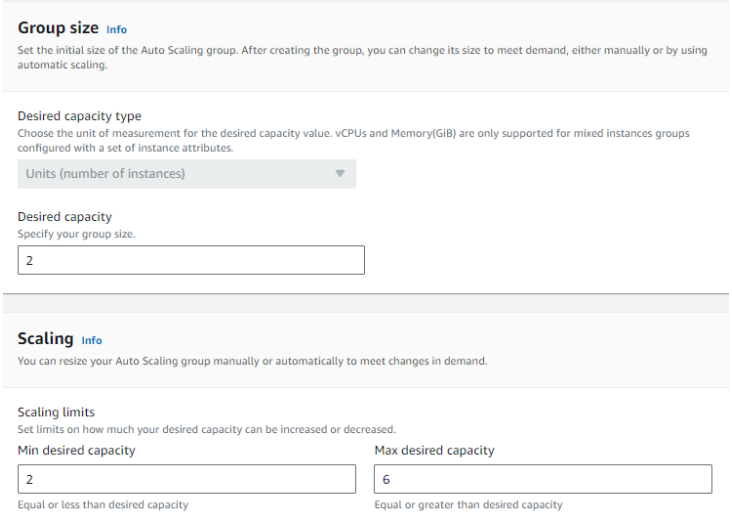
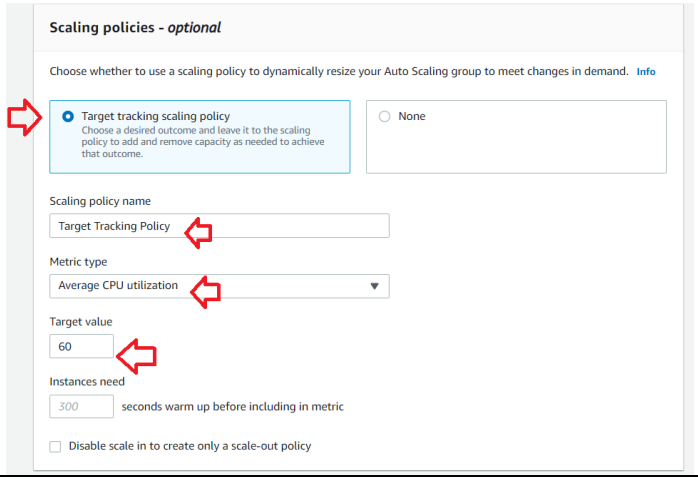
SN	Steps	Instructions

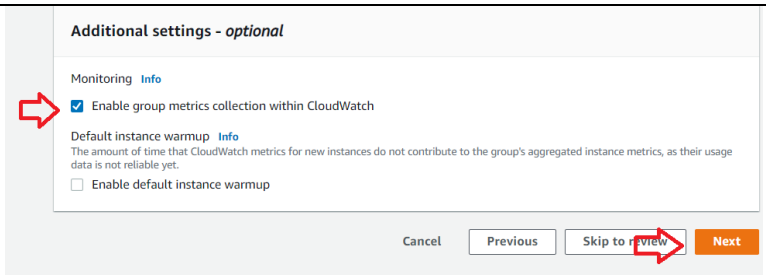
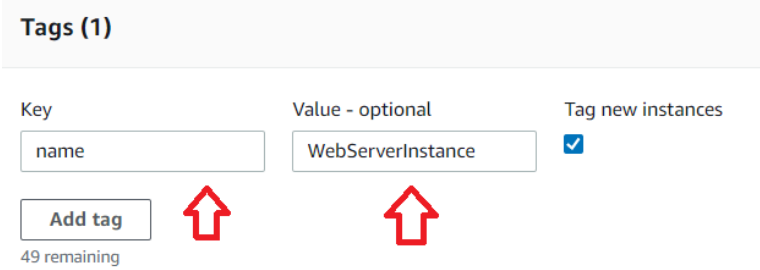
1)	Click **Create a launch template**	
2)	Name the launch template	
3)	Choose Amazon machine image (AMI): **WebServerAMI**	
4)	Choose Instance Type: t2.micro (free tier)	
5)	Key pair	Choose an existing key pair
6)	Network settings	No need to specify the subnet and use the default setting. 

		Use an existing security group which allows SSH, HTTP, and HTTPS traffic. Click **Create launch template**

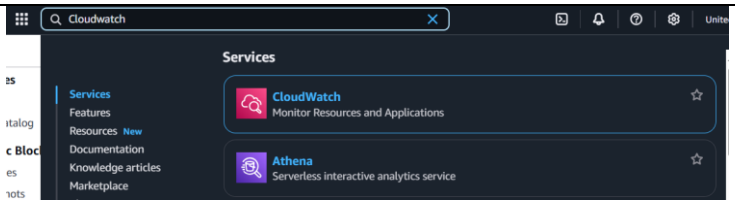
3. Create an Auto Scaling group.

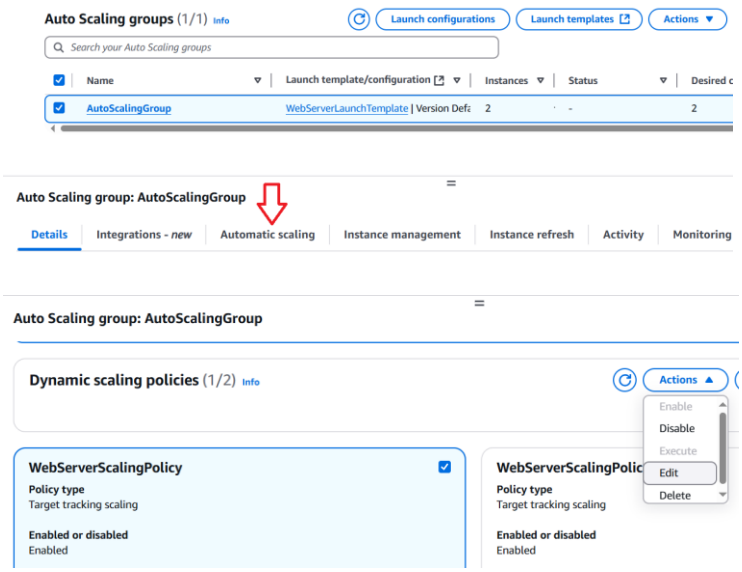
SN	Steps	Instructions
1)	Create and name the autoscaling group as **WebServerAutoScalingGroup**. Choose VPC and the availability zones and subnets and click **Next**	 Choose the default VPC and select at least two subnets. 
3)	Load balancing: Attach to an existing load balancer	Select Attach to an existing load balancer, and choose WebServerGroup as the load balancer target group.

		
4)	Configure group size and scaling policies	
5)	Configure Scaling policies and click **Next**	Scaling policies – optional: choose **Target tracking scaling policy** and configure: ○ Lab policy name: **WebServerScalingPolicy** ○ Metric type: **Average CPU Utilization** ○ Target value: **60** 

6)	Enable group metrics collection within CloudWatch and click **Next**	
7)	Add notifications	Use the default setting that the system will send notifications to SNS topics whenever Amazon EC2 Auto Scaling launches or terminates the EC2 instances in your Auto Scaling group and click **Next.**
8)	Add tags and create an auto scaling group	 click **Next**
9)	Review and create the auto scaling group	Click **Create an Auto Scaling group**

Task 4: Test Auto Scaling

SN	Steps	Instructions
1)	Monitor the performance using CloudWatch	 On the **Services** menu, click **CloudWatch**. In the left navigation pane, choose **All alarms**. Two alarms will be displayed. These were created automatically by the Auto Scaling group. They will automatically keep the average CPU load close to

		60% while also staying within the limitation of having two to six instances. The two alarms: one is xxxx-**AlarmLow**-xxxx which will generate alarm if the CPU utilisation is below the low threshold, and the other alarm is xxxx-**AlarmHigh**-xxxx which should show **OK** which indicates the CPU utilisation is not beyond the high threshold and the alarm has not been triggered. The chart should show very low levels of CPU utilisation now. It may take about 3-5 minutes for the system to collect sufficient data.
2)	Update the Target Value for auto scaling	On the **Services** menu, click **EC2**. In the left navigation pane, choose **Auto Scaling Groups**, select **WebServerAutoScalingGroup**. In the bottom half of the page, choose the **Automatic Scaling** tab, choose **Target Tracking Policy**, Click Actions and click **Edit**. Change the Target Value to **0.05**, click **Update**.  In less than 5 minutes, the **AlarmLow** alarm should change to **OK** and the **AlarmHigh** alarm status should change to **In alarm**. You can click Refresh in the top-right every 60 seconds to update the display. You should see the AlarmHigh chart indicating an increasing CPU percentage. Once it crosses the 0.05% line for more than 3

		minutes, it will trigger Auto Scaling to add additional instances. On the Services menu, click **EC2**. In the left navigation pane, click **Instances**. There are a few new instance(s) which were created by Auto Scaling in response to the Alarm. **Add screenshot of CloudWatch alarms and running EC2 to Result 3 in the report.**
3)	Change back Target Value for auto scaling	On the **Services** menu, click **EC2**. In the left navigation pane, choose **Auto Scaling Groups**, select **WebServerAutoScalingGroup**. In the bottom half of the page, choose the **Automatic Scaling** tab, choose **Target Tracking Policy**, Click Actions and click **Edit**. Change the Target Value to **60**, click **Update**. After about 3 to 5 minutes, the **AlarmHigh** alarm will change from **In alarm** to **OK**, some newly added instances will be stopped and only 2 instances will be running. **Add screenshot of CloudWatch alarms and running EC2 to Result 3 the report.**

Task 5: Terminate the services

SN	Steps	Instructions
1)	Terminate **Web Service** instances	Go to **Instances** in the left navigation pane, check **Web Server** instances, choose Instance state menu, select **Terminate Instance**. Similarly, delete all the running instances initiated by auto scaling.
2)	Delete Load Balancer	Go to **Load Balancers** in the left navigation pane under **Load Balancing** section, select **WebServerELB,** select the **Actions** menu, choose

		Delete load balancer.
3)	Delete Auto Scaling	Go to **Auto Scaling Groups** in the left navigation pane under **Auto Scaling** section, select **WebServerAutoScalingGroup** and choose **Delete** Go to **Launch Configurations** in the left navigation pane under **Auto Scaling** section, check **WebServerConfig,** select the **Actions** menu, choose **Delete launch configuration**. Go to **Target Groups** in the left navigation pane under **Load Balancing** section, check **WebServerGroup,** select the **Actions** menu, choose **Delete.**
5)	Delete AMI	Go to **AMI** in the left navigation pane under **Images** section, check **WebServerAMI**, select the **Actions** menu, choose **Deregister AMI.** Go to **Snapshots** in the left navigation pane under **Elastic Block Store** section, select **Web Server AMI**. In the **Actions** menu, choose **Delete snapshot.** Go to **Volumes** under Elastic Block Store in the left pane, delete the volume associated with the web server instances created by the AutoScaling if any**.**

Add one or two sentences to summarize what you have learned from the lab into the lab report.

Lab complete

Congratulations! You have completed the lab.

Please complete the lab report and submit it to the x-site by end of 17 Mar 2026 (Tuesday).

Thank you!